AI Assistant Coding
Lab - 3.3

Task 1

Zero-Shot Prompting – Leap Year Check
Scenario
Zero-shot prompting involves giving instructions without providing examples.
Task Description
Use zero-shot prompting to instruct an AI tool to generate a Python function that:
• Accepts a year as input
• Checks whether the given year is a leap year
• Returns an appropriate result
Note: No input-output examples should be provided in the prompt.
Expected Output
• AI-generated leap year checking function
• Correct logical conditions
• Sample input and output
• Screenshot of AI-generated response (if required)

Prompt

```
#write a program to accept a year as input and check whether it is a
leap year or not
```

Code

```
#write a program to accept a year as input and check whether it is a
leap year or not
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
#input year
year = int(input("Enter a year: "))
if is_leap_year(year):
    print(f"{year} is a leap year.")
else:
    print(f"{year} is not a leap year.")
```

Output

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab3.py"
Enter a year: 2024
2024 is a leap year.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis

The AI generated a correct leap year function using standard conditional logic without needing examples. This shows that zero-shot prompting works well for common, well-defined problems.

Task 2

One-Shot Prompting – Centimeters to Inches Conversion
Scenario
One-shot prompting guides AI using a single example.
Task Description
Use one-shot prompting by providing one input-output example to generate a Python function that:
• Converts centimeters to inches
• Uses the correct mathematical formula
Example provided in prompt:
Input: 10 cm → Output: 3.94 inches
Expected Output
• Python function with correct conversion logic
• Accurate calculation
• Sample test cases and outputs

Prompt

```
#take user input to convert centimetres to inches use correct
mathematical formula 10cm = 3.93701 inches
```

Code

```
#take user input to convert centimetres to inches use correct
mathematical formula 10cm = 3.93701 inches
def cm_to_inches(cm):
    inches = cm * 0.393701
    return inches
#input centimeters
cm = float(input("Enter length in centimeters: "))
inches = cm_to_inches(cm)
print(f"{cm} cm is equal to {inches} inches.")
#example input 10 output 3.93701
```

```
cm = 10
inches = cm_to_inches(cm)
print(f"{cm} cm is equal to {inches} inches.")
```

Output

```
Enter length in centimeters: 25.4
25.4 cm is equal to 10.0000054 inches.
10 cm is equal to 3.9370100000000003 inches.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis

Providing one example helped the AI apply the correct conversion formula accurately.
The generated function was simple, readable, and produced precise results.

Task 3

Few-Shot Prompting – Name Formatting
Scenario
Few-shot prompting improves accuracy by providing multiple examples.
Task Description
Use few-shot prompting with 2–3 examples to generate a Python function that:
• Accepts a full name as input
• Formats it as “Last, First”
Example formats:
• "John Smith" → "Smith, John"
• "Anita Rao" → "Rao, Anita"
Expected Output
• Well-structured Python function
• Output strictly following example patterns
• Correct handling of names
• Sample inputs and outputs

Prompt

```
#accept a full name from user and format it into last, first  "John
Smith" → "Smith, John" "Anita Rao" → "Rao, Anita
```

Code

```
#accept a full name from user and format it into last, first  "John
Smith" → "Smith, John" "Anita Rao" → "Rao, Anita
def format_name(full_name):
    parts = full_name.split()
    if len(parts) >= 2:
        first_name = " ".join(parts[:-1])
```

```
        last_name = parts[-1]
        return f"{last_name}, {first_name}"
    else:
        return full_name
#input full name
full_name = input("Enter your full name: ")
formatted_name = format_name(full_name)
print(f"Formatted name: {formatted_name}")
#example input "Anita Rao" output "Rao, Anita" give output for this
full_name = "Anita Rao"
formatted_name = format_name(full_name)
print(f"Formatted name: {formatted_name}")
```

Output

Analysis

Multiple examples guided the AI to strictly follow the required output format. Few-shot prompting improved consistency and correct handling of different names.

Task 4

Comparative Analysis – Zero-Shot vs Few-Shot
Scenario
Different prompt strategies may produce different code quality.
Task Description
• Use zero-shot prompting to generate a function that counts vowels in a string
• Use few-shot prompting for the same problem
• Compare both outputs based on:
o Accuracy
o Readability
o Logical clarity
Expected Output
• Two vowel-counting functions
• Comparison table or short reflection paragraph
• Conclusion on prompt effectivenes

Prompt - Zero Shot

```
#write a program to count vowels in a string
```

Code - Zero Shot

```
#write a program to count vowels in a string
def count_vowels(input_string):
    vowels = "aeiouAEIOU"
    count = sum(1 for char in input_string if char in vowels)
    return count
#input string
input_string = input("Enter a string: ")
vowel_count = count_vowels(input_string)
print(f"The number of vowels in the string is: {vowel_count}")
```

Output

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab3.py"
Enter a string: hello world
The number of vowels in the string is: 3
PS C:\Users\dasar\AI-Assistant-Coding>
```

Prompt - Few Shot

```
#write a program to count vowels in a string for example input : "Hello
World" output : 3
```

Code - Few Shot

```
#write a program to count vowels in a string for example input : "Hello
World" output : 3
def count_vowels(input_string):
    vowels = "aeiouAEIOU"
    count = sum(1 for char in input_string if char in vowels)
    return count
#example input
input_string = "Hello World"
vowel_count = count_vowels(input_string)
print(f"The number of vowels in the string is: {vowel_count}.")
#output : 3
```

Output

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab3.py"
The number of vowels in the string is: 3.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis

Both zero-shot and few-shot approaches produced correct vowel-counting logic. However, few-shot prompting resulted in clearer output alignment with the given example.

Criteria	Zero-Shot	Few-Shot
Accuracy	Correct vowel count	Correct and example-matched
Readability	Simple and clean	Clearer due to example
Logical Clarity	Logic is straightforward	Logic is easy to follow

Task 5

Few-Shot Prompting – File Handling
Scenario
File processing requires clear logical understanding.
Task Description
Use few-shot prompting to generate a Python function that:
• Reads a .txt file
• Counts the number of lines in the file
• Returns the line count
Expected Output
• Working Python file-processing function
• Correct line count
• Sample .txt input and output
• AI-assisted logic explanation

Prompt

```
#generate a python program to read a text.file and count the number of
lines in the file and return the count
```

Code

```
#generate a python program to read a text.file and count the number of
lines in the file and return the count
def count_lines_in_file(file_path):
    try:
        with open(file_path, 'r') as file:
            lines = file.readlines()
            return len(lines)
    except FileNotFoundError:
        print("The file was not found.")
        return 0
#input file path
file_path = input("Enter the path to the text file: ")
line_count = count_lines_in_file(file_path)
print(f"The number of lines in the file is: {line_count}.")
#example input : "sample.txt" output : 10
file_path = r"C:\Users\dasar\OneDrive\Documents\sample.txt"
line_count = count_lines_in_file(file_path)
print(f"The number of lines in the file is: {line_count}.")
#output : 10
```

Output

```
PS C:\Users\dasar\AI-Assistant-Coding> python -u "c:\Users\dasar\AI-Assistant-Coding\lab3.py"
Enter the path to the text file: C:\Users\dasar\OneDrive\Documents\syllabus.txt
The number of lines in the file is: 5.
The number of lines in the file is: 10.
PS C:\Users\dasar\AI-Assistant-Coding>
```

Analysis

Examples helped the AI generate a robust file-handling function with error handling. The code correctly counted lines and handled missing files safely.